The `choices` field configuration option is a `Model`-level validation that provides a set of values the field can contain. This option takes an iterable of pairs, usually written as a list of tuples. Each pair specifies the value stored in the database and a display string to go with it. For example, a field containing a country code could specify `choices=[("US", "United States"), ("CA", "Canada"), ("MX", "Mexico")]`, if you only supported countries in North America. When this field gets shown in the Django Admin or in a web form, a `<select>` drop-down box gets used, with the full string of the country's name visible to the user and the resulting value for the `<select>` being the corresponding two-letter country code.

It is common practice to build an attribute into your `Model` class that specifies the choices value. An `Address` class could contain a `COUNTRY_CHOICES` attribute specifying the previous example's listing. The `choices` option would then point to the attribute. Doing it this way makes the list of choices available anywhere the model gets imported. There are third-party libraries that define frequently used tuple sets. For example, the "django-localflavor" (https://pypi.org/project/django-localflavor/) package includes submodules for many countries and choices-compatible listings for states and provinces within them.

Databases improve look-up performance through the implementation of a special data structure known as an *index*. Indexes are similar to dictionaries, in that they provide a quick way of looking up a given value based on a key, rather than scanning an entire list. The ID field of your Django `Model` is indexed by default. This ensures that `.get(id=obj_id)` performs quickly. If a field is going to be frequently used to look up a row, that field may need an index. For example, if you add search capabilities to your site allowing a musician to be found by last name, adding an index to the `last_name CharField` will make a significant performance improvement. The `db_index` field option is a Boolean; setting it to `True` causes Django to add an index on the corresponding column.

There are times when you want the value of a field to be unique within a table. Django provides several field options to enforce this: `unique`, `unique_for_date`, `unique_for_month`, and `unique_for_year`. The `unique` field operates at both the `Model` validation level and applies a constraint to the database. When `unique=True`, only one object in the table will be allowed to have the value found in that field. Any attempt to save another object with the same value will result in an exception. Django automatically applies an index to this column, as it needs to look values up on every save of an object to ensure uniqueness. The `unique_for_date`, `unique_for_month`, and `unique_for_year` options are special cases for `DateTimeFields`. The uniqueness constraint gets applied to the date, month, or year portion of the field, while the other parts (e.g., the timestamp) are not considered.

4.8.2 Popular fields

The field you've used so far is probably the most popular one out there: `CharField`. There are several variations on this field that are essentially text storage mechanisms but with different validation and default values. The `EmailField`, `URLField`, and